



# Ansible常用模块

誉天教育

- file模块
- copy模块
- fetch模块
- yum\_repository模块
- yum模块
- service模块
- systemd模块
- cron模块
- user模块
- group模块

- get\_url 模块
- unarchive模块
- synchronize模块



- file模块主要用于远程主机上的文件操作，file模块包含如下选项：

- ◆ path：必选项，定义文件/目录的路径
- ◆ state：

file：查看文件状态，默认选项，若文件不存在，也不会被创建，会报错文件不存在

touch：如果文件不存在，则会创建一个新的文件，如果文件或目录已存在，则更新时间戳

absent：删除目录、文件或者取消链接文件

directory：如果目录不存在，创建目录

link：创建软链接

hard：创建硬链接

- ◆ owner：定义文件/目录的属主
- ◆ group：定义文件/目录的属组
- ◆ mode：定义文件/目录的权限
- ◆ src：要被链接的源文件的路径，只应用于state=link的情况
- ◆ dest：被链接到的路径，只应用于state=link的情况
- ◆ force：需要在两种情况下强制创建软链接，一种是源文件不存在但之后会建立的情况下；另一种是目标软链接已存在，需要先取消之前的软链，然后创建新的软链，有两个选项：yes|no

- 使用示例

- ◆ `ansible 192.168.8.120 -m file -a 'path=/tmp/test.txt state=file'`
- ◆ `ansible 192.168.8.120 -m file -a 'path=/tmp/test.txt state=touch owner=root group=root mode=644'`
- ◆ `ansible 192.168.8.120 -m file -a 'src=/tmp/test.txt dest=/root/test.txt state=link'`
- ◆ `ansible 192.168.8.120 -m file -a 'path=/tmp/test state=directory owner=root group=root mode=755'`
- ◆ `ansible 192.168.8.120 -m file -a 'path=/tmp/test2/test3/aaa/bbb state=directory owner=root group=root mode=755'`
- ◆ `ansible 192.168.8.120 -m file -a 'path=/tmp/test2 state=absent'`

- 复制文件到远程主机，copy模块包含如下选项：
  - ◆ src：要复制到远程主机的文件在本地的地址，可以是绝对路径，也可以是相对路径。如果路径是一个目录，它将递归复制。在这种情况下，如果路径使用"/"来结尾，则只复制目录里的内容，如果没有使用"/"来结尾，则包含目录在内的整个内容全部复制，类似于rsync。
  - ◆ dest：必选项。要将源文件复制到的远程主机的绝对路径。
  - ◆ force：如果目标主机包含该文件，但内容不同，如果设置为yes，则强制覆盖，如果为no，则只有当目标主机的目标位置不存在该文件时，才复制。默认为yes
  - ◆ backup：在覆盖之前将原文件备份，备份文件包含时间信息。有两个选项：yes|no
  - ◆ content：用于替代“src”，可以直接设定指定文件的值
  - ◆ remote\_src：复制远程主机的文件

- 使用示例：

- ◆ `ansible 192.168.8.120 -m copy -a 'src=/data dest=/usr/local/src/'`
- ◆ `ansible 192.168.8.120 -m copy -a 'src=/data/ dest=/usr/local/src/'`
- ◆ `ansible 192.168.8.120 -m copy -a 'src=/etc/ansible/ansible.cfg dest=/usr/local/src/ owner=root group=root mode=644'`
- ◆ `ansible 192.168.8.120 -m copy -a 'backup=yes src=/etc/fstab dest=/usr/local/src/ansible.cfg owner=root group=root mode=644'`
- ◆ `ansible 192.168.8.120 -m copy -a 'content="just a test!" dest=/usr/local/src/test.txt'`

- fetch模块与copy模块正好相反，copy是把主控端的文件复制到被控端，而fetch则是把被控端的文件复制到主控端。并且在主控端指定的目录下，以被控端主机名的形式来组织目录结构。常用选项有：
  - ◆ src：被控端要复制的文件地址，只能是文件，不能是目录
  - ◆ dest：主控端存放文件的地址
  - ◆ flat：默认为no，表示在主控端目录下以被控端主机名的形式来组织目录结构。yes表示不会以被控端主机名的形式来组织目录结构，dest必须以/结尾。
- 使用示例：
  - ◆ `ansible node1 -m fetch -a 'src=/etc/shadow dest=test'`
  - ◆ `ansible node1 -m fetch -a 'src=/etc/shadow dest=test/ flat=yes'`

- 使用yum\_repository管理yum仓库，其选项有：
  - ◆ file：配置文件的名字，不包含.repo
  - ◆ name：yum仓库的名字
  - ◆ description：仓库的描述信息
  - ◆ baseurl：yum源地址
  - ◆ enabled：是否开启yum仓库，yes/on
  - ◆ gpgcheck：是否检查软件包的完整性，yes/no
  - ◆ gpgkey：公钥地址
- 使用示例
  - ◆ `ansible node1 -m yum_repository -a 'file=media name=BaseOS description= BaseOS baseurl=file:///media/BaseOS enabled=yes gpgcheck=yes gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release'`

- 使用yum包管理器来管理软件包，其选项有：
  - ◆ name：要进行操作的软件包的名字，也可以传递一个url或者一个本地的rpm包的路径
  - ◆ state：状态（present安装，absent卸载，latest最新版本）
- 使用示例
  - ◆ `ansible test -m yum -a 'name=httpd state=latest'`
  - ◆ `ansible test -m yum -a 'name=httpd state=absent'`
  - ◆ `ansible test -m yum -a 'name="@Development Tools" state=present'`

- 用于管理服务，该模块包含如下选项：
  - ◆ name：必选项，服务名称
  - ◆ state：对当前服务执行启动，停止、重启、重新加载等操作（started,stopped,restarted,reloaded）
  - ◆ enabled：是否开机启动 yes|no
- 使用示例：
  - ◆ `ansible test -m service -a "name=httpd state=started enabled=yes"`

- 用于管理服务，该模块包含如下选项：

- ◆ name：指定服务名称
- ◆ state：管理服务状态，reloaded|restarted|started|stopped
- ◆ enabled：是否设置开机自启
- ◆ daemon\_reload：当服务配置文件发生变更时重载服务

- 使用示例：

- ◆ `ansible test -m systemd -a "name=httpd state=started enabled=yes daemon_reload=yes "`

- 用于管理计划任务，包含如下选项：
  - ◆ name：该任务的描述
  - ◆ state：确认该任务计划是创建还是删除，present创建，absent删除
  - ◆ user：以哪个用户的身份执行
  - ◆ day：日（1-31，，/2，.....）
  - ◆ hour：小时（0-23，，/2，.....）
  - ◆ minute：分钟（0-59，，/2，.....）
  - ◆ month：月（1-12，，/2，.....）
  - ◆ weekday：周（0-7，\*，.....）
  - ◆ job：要执行的任务，依赖于state=present
  - ◆ cron\_file：指定配置文件，例如/etc/crontab，/etc/cron.d/\*

- 使用示例：

- ◆ `ansible test -m cron -a 'name="a job for reboot" hour=12 minute=0 job="reboot"'`
- ◆ `ansible test -m cron -a 'name="yum autoupdate" weekday="2" minute=0 hour=12 user="root"'`
- ◆ `ansible test -m cron -a 'name="test" minute="0" hour="5,2" job="ls -alh > /dev/null"'`
- ◆ `ansible test -m cron -a 'name="testcron minute" minute="*/10" job="echo test >> /tmp/test"'`
- ◆ `ansible test -m cron -a 'name="testcron minute" state=absent'`
- ◆ `ansible node1 -m cron -a 'name="testcron minute" user=root state=present job="echo test >> /tmp/test" cron_file=/etc/crontab'`

- user模块请求的是useradd, userdel, usermod 三个指令：
  - ◆ state：是创建还是删除
  - ◆ name：指定用户名
  - ◆ uid：指定用的uid
  - ◆ group：指定用户属组
  - ◆ groups：指定用户的附加组
  - ◆ comment：定义用户描述信息
  - ◆ create\_home：是否创建家目录 yes|no
  - ◆ home：指定用户的家目录，需要与createhome配合使用
  - ◆ shell：指定用户的shell环境
  - ◆ password：指定用户的密码
  - ◆ remove：当state=absent时，remove=yes则表示连同家目录一起删除，等价于userdel -r

- 使用示例：

- ◆ `ansible test -m user -a 'name=johnd comment="John Doe" uid=1040 group=admin'`
- ◆ `ansible test -m user -a 'name=james shell=/bin/bash groups=admins,developers append=yes'`
- ◆ `ansible test -m user -a 'name=johnd state=absent remove=yes'`

#需要说明的是，在指定password参数时，不能使用明文密码，因为后面这一串密码会被直接传送到被管理主机的/etc/shadow文件中，所以需要先将密码字符串进行加密处理。然后将得到的字符串放到password中即可。

#使用下面的密码创建用户

```
# echo "yutian" | openssl passwd -1 -salt $(< /dev/urandom tr -dc '[:alnum:]' | head -c 32) -stdin
```

```
$1$4P4PIFuE$ur9ObJiT5iHNrb9QnjaIB0
```

- ◆ `ansible all -m user -a 'name=foo password="$1$4P4PIFuE$ur9ObJiT5iHNrb9QnjaIB0"'`

- group模块请求的是groupadd, groupdel, groupmod 三个指令：
  - ◆ gid：指定组id
  - ◆ name：指定组名
  - ◆ state：创建还是删除组，选项：present|absent
- 使用示例：
  - ◆ `ansible test -m group -a 'name=test gid=1001 state=present'`

- 该模块主要用于从http、ftp、https服务器上下载文件（类似于wget），主要有如下选项：
  - ◆ url：下载的URL
  - ◆ url\_password、url\_username：主要用于需要用户名密码进行验证的情况
  - ◆ dest：目标目录
  - ◆ mode：权限
  - ◆ owner：拥有人
  - ◆ group：拥有组
- 使用示例：
  - ◆ `ansible node1 -m get_url -a 'url=https://mirrors.aliyun.com/centos/8/BaseOS/x86_64/os/Packages/samba-4.9.1-8.el8.x86_64.rpm dest=/tmp/samba.rpm mode=0440'`

- 用于解压文件，模块包含如下选项：
  - ◆ src：则需要指定压缩文件的源路径
  - ◆ dest：远程主机上的一个路径，即文件解压的路径
  - ◆ remote\_src：如果为yes，则文件会从主控端复制到被控端。否则会直接尝试从被控端查找文件。默认为yes。
  - ◆ owner：解压后文件或目录的属主
  - ◆ group：解压后的目录或文件的属组
  - ◆ mode：解压后文件的权限
  - ◆ creates：指定一个文件名，当该文件存在时，则解压指令不执行
  - ◆ list\_files：如果为yes，则会列出压缩包里的文件，默认为no

- 使用示例：
  - ◆ `ansible node1 -m unarchive -a 'src=etc.tgz dest=/tmp/etc/'`
  - ◆ `ansible node1 -m unarchive -a 'src=https://example.com/example.zip dest=/usr/local/bin remote_src=yes'`

- 使用rsync同步文件，其参数如下：
  - ◆ src：要复制的文件，目录以/结尾表示包含目录本身，目录不以/结尾表示不包含目录本身
  - ◆ dest：目录路径
  - ◆ archive: 归档，相当于同时开启recursive(递归)、links、perms、times、owner、group、-D选项都为yes，默认该项为开启
  - ◆ compress：是否开启压缩
  - ◆ rsync\_opts：rsync的选项，选项之间用逗号隔开，例如-a,-v,-z等等
  - ◆ delete: 删除不存在的文件，默认no
  - ◆ dest\_port：默认目录主机上的端口，默认是22，走的ssh协议
  - ◆ mode: push或pull 模块，push模式的话，一般用于从本机向远程主机上传文件，pull 模式用于从远程主机上取文件，默认是push。

- 使用示例：

- ◆ `ansible node1 -m synchronize -a 'src=/etc dest=/tmp/'`
- ◆ `ansible node1 -m synchronize -a 'src=/etc dest=/tmp/ archive=no rsync_opts=-r links=yes'`

- push 在主控端执行

- ◆ `ansible node1 -m synchronize -a 'archive=yes compress=yes dest=/data/ src=/data/ rsync_opts=-vz mode=push'`
- ◆ 以上等同于 `rsync -avz /data/* 192.168.40.131:/data/`

- push 在被控端执行

- ◆ `ansible node1 -m synchronize -a 'archive=yes compress=yes dest=/data/ src=/data/ rsync_opts=-vz mode=pull'`
- ◆ 以上等同于 `rsync -avz 192.168.40.130:/data/* /data/`

- ✓ 常用模块的使用

